



4



16



1920 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

Conflict Resolution

د. عمر حمدون

محتوى مجاني غير مخصص للبيع

نظم قواعد المعرفة

RB Informatcs; 07/05/202



محرك الاستنتاج (Inference Engine)

محرك الاستنتاج يسمى أحياناً المفسر (Interpreter)، وهو جزء من الأنظمة الذكية. وظيفته هي استخدام المعرفة الموجودة داخل النظام لإجراء استدلالات (Reasoning) واتخاذ قرارات بناءً على القواعد.

كيف يعمل محرك الاستنتاج؟

1. التحقق من القواعد:

- عندما يحتاج النظام لاتخاذ قرار، يقوم محرك الاستنتاج بمقارنة الشرط (condition) في القاعدة مع البيانات المخزنة في الذاكرة (Working Memory - WM).
- إذا كانت البيانات المخزنة تتطابق مع الشرط، فإن القاعدة تصبح قابلة للتنفيذ (أو "تطلق Fire").

2. تنفيذ القاعدة:

- إذا كانت القاعدة قابلة للتنفيذ (أي أن الشروط في القاعدة صحيحة)، فإن النظام سيقوم بتنفيذ الإجراء (action) المرفق بها.

مثال على فحص القاعدة:

افترض أن لديك القاعدة التالية:

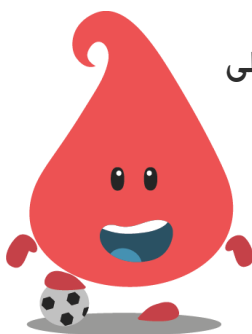
- القاعدة 1: إذا كان الطقس بارداً، فقم بارتداء معطف.
- القاعدة 2: إذا كان الطقس ممطراً، فقم باستخدام مظلة.

الخطوة الأولى: يقوم محرك الاستنتاج بمقارنة الشروط في القواعد مع البيانات المخزنة في الذاكرة مثل "الطقس بارد" و"الطقس ممطر".

الخطوة الثانية: بناءً على هذه المقارنة، قد يتم إطلاق (Firing) القاعدة التي تتوافق مع البيانات.

ماذا يحدث إذا كانت هناك عدة قواعد؟

في بعض الأحيان، يمكن أن تتطابق أكثر من قاعدة مع البيانات الموجودة. هنا، يجب على محرك الاستنتاج أن يقرر أي قاعدة سيطبق أولاً.



استراتيجية حل التضارب (Conflict Resolution Strategy)

حل التضارب هو طريقة يستخدمها محرك الاستنتاج لاختيار القاعدة الأفضل عندما تتطابق أكثر من قاعدة.

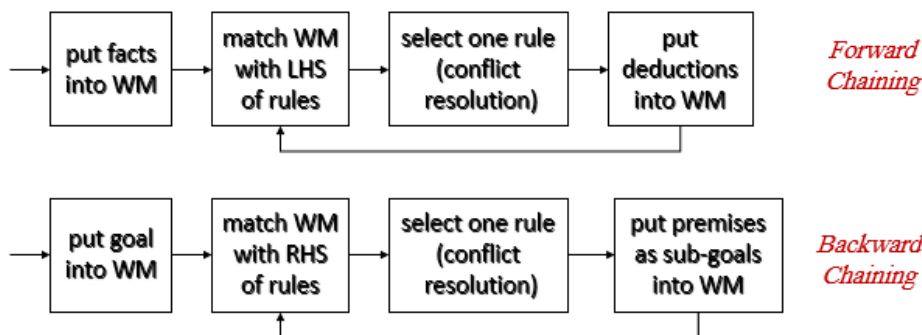
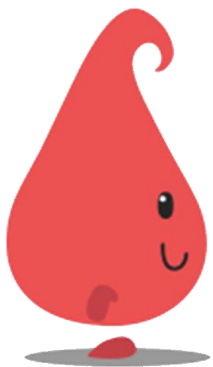
مثلاً:

- اختيار القاعدة التي تم تعريفها أولاً.
- أو اختيار القاعدة التي تحتوي على أكثر الشروط تطابقاً.

أي محرك الاستنتاج:

- ❖ يتحقق من القواعد باستخدام البيانات المخزنة.
- ❖ يحدد أي القواعد ستُطبق.
- ❖ إذا تطابقت أكثر من قاعدة، يستخدم استراتيجية لحل التضارب لاختيار القاعدة الأنسب.

Block Diagrams of Forward and Backward Chaining



What to do if there is more than 1 matching rule in each inference cycle?

CONFLICT RESOLUTION

ليكن لدينا هذه القواعد لمثال لعبور الطريق:

Rule 1:

If the 'traffic light' is green
THEN the action is go

Rule 2:

IF the 'traffic light' is red
THEN the action is stop

Rule 3:

IF the 'traffic light' is red
THEN the action is go

conflict



في هذا المثال، لدينا قواعد مرتبطة بإشارات المرور التي تحدد ما يجب فعله عند عبور الطريق. لنفترض أن هناك ثلاث قواعد:

كيف يعمل محرك الاستنتاج في هذا المثال؟

1. التحقق من الشروط (Matching) :

- في البداية، يقوم محرك الاستنتاج بمقارنة الشروط (الجزء الأول من القواعد) مع البيانات الموجودة في قاعدة البيانات.
- إذا كانت البيانات تتطابق مع الشرط، فإن القاعدة تصبح قابلة للتنفيذ (يتم "إطلاق" القاعدة).

2. التحقق من القواعد المتعارضة:

- في هذه الحالة، القاعدة 2 و القاعدة 3 لديهما نفس الشرط: "إذا كان إشارة المرور حمراء."
- لكن الفعل (THEN part) يختلف: القاعدة 2 تقول "توقف"، بينما القاعدة 3 تقول "اذهب".

3. إطلاق القواعد:

- إذا كانت إشارة المرور حمراء، فإن القاعدة 2 و القاعدة 3 يمكن أن تنطلق لأن الشرط في كلا القاعدتين يتطابق مع "إشارة المرور حمراء."
- لكن، يجب أن يقرر محرك الاستنتاج أي قاعدة يتم تنفيذها أولاً.

4. استراتيجية حل تضارب (Conflict Resolution) :

عندما تتوافر أكثر من قاعدة تنطبق في نفس الوقت، يجب أن يقرر النظام أي القواعد سيُنفذ أولاً. في هذه الحالة، لدينا تضارب بين القاعدتين 2 و 3.

- في الترتيب الطبيعي للقواعد:
- القاعدة 2 (توقف) سيتم تنفيذها أولاً لأنها مكتوبة أولاً في النظام.
- بعد ذلك، القاعدة 3 (اذهب) سيتم تنفيذها أيضاً لأن شرطها (إشارة المرور حمراء) يتطابق.

التفسير خطوة بخطوة:

1. إشارة المرور حمراء:

- القاعدة 2 تنطبق أولاً (توقف).
- القاعدة 3 أيضاً تنطبق (اذهب).
- القاعدة 2 تنفذ أولاً، وبالتالي يتم تنفيذ الفعل "توقف".
- القاعدة 3 تنفذ أيضاً، ويتم تنفيذ الفعل "اذهب" (لكن هذا غير منطقي لأننا لا يمكننا أن نتوقف ونذهب في نفس الوقت).

الحل:

- من الواضح أن ترتيب القواعد في قاعدة المعرفة مهم جداً.



- في هذا المثال، استراتيجية حل التصادم يمكن أن تكون تحديد الهدف، أي أن محرك الاستنتاج يجب أن يتوقف عند القاعدة التي تحقق الهدف الأول (التوقف عند الضوء الأحمر).
- إذا كان الهدف هو أن نقول "توقف" أولاً، نحتاج إلى ترتيب القواعد بحيث تكون القاعدة 2 (التوقف) تأتي قبل القاعدة 3 (انذهب).

استراتيجيات حل التصادم (Conflict Resolution Strategies)

عند تصميم نظام قائم على القواعد (Rule-Based System)، يجب الانتباه إلى الآلية التي تُنفَّذ بها القواعد، خاصة إذا كانت هناك عدة قواعد قابلة للتفعيل في نفس الوقت. من بين استراتيجيات التحكم في تفعيل القواعد، نجد مجموعة من المعايير التي تُستخدم لتحديد أولوية التنفيذ. أهم هذه المعايير:

1. Textual Order الترتيب النصي للقواعد

- المقصود به أن القواعد في قاعدة المعرفة (Knowledge Base) تُرتَّب بشكل خطي (سطراً بعد سطر).
- عند تنفيذ النظام، يتم فحص القواعد بهذا الترتيب، ويتم تفعيل أول قاعدة ينطبق شرطها.
- لذلك، يُنصح بوضع القواعد الأكثر أهمية في البداية، لضمان تفعيلها قبل غيرها.

مثال:

لدينا القواعد التالية:

- R1:** IF A THEN E
- R2:** IF B & C & D THEN F
- R3:** IF E & F THEN G
- R4:** IF G THEN X
- R5:** IF F THEN H
- R6:** IF H THEN Y

إذا كان الشرط A محققاً، فإن R1 ستُنفَّذ أولاً. وفي الدورة التالية، ستعود R1 وتُفَعَّل مجدداً، ما لم نستخدم مفهوم الجمود (Refractoriness).

2. Refractoriness الجمود أو منع التكرار

- يُستخدم لمنع القاعدة من أن تُنفَّذ مرة أخرى إذا سبق تفعيلها.
- يُفيد في منع الحلقات اللانهائية أو التكرار غير الضروري.
- في مثالنا: بعد تنفيذ R1، سيتم تجاهلها في الدورات التالية إذا كان النظام يستخدم خاصية "الجمود".

3. Recency

- يُفضل القواعد التي تعتمد على بيانات تم إنتاجها مؤخراً.
- الفرضية هنا: "المعلومة الأحدث هي الأكثر صلة بالوضع الحالي".



4. Breadth الاتساع

- تُفضّل القواعد التي تم تفعيلها لفترة طويلة على القواعد المفعلة حديثاً. أي أن القواعد التي تطابق الحقائق القديمة تُفضّل على القواعد التي تطابق الحقائق الجديدة، تميل إلى البحث بطريقة تركز على الاتساع.

5. Specificity التخصص

- تفضل القواعد التي تحتوي على شروط أكثر تحديداً.
- تستخدم عندما نريد الوصول إلى استنتاجات دقيقة جداً بناءً على معطيات محددة.

6. Simplicity البساطة

- تُعطي الأفضلية للقواعد ذات الشروط الأبسط (مثلاً شرط واحد بدلاً من ثلاثة).
- هذا يُسرّع عملية التحقق ويُبسّط فهم القاعدة.

7. LEX (Least recently used / rule depth)

- اختصار لـ "Linear EXecution" أو "Least EXecuted" بحسب السياق.
- تُعطي الأولوية للقواعد التي لم تستخدم منذ فترة طويلة أو الموجودة في أعماق شجرة الاستدلال.

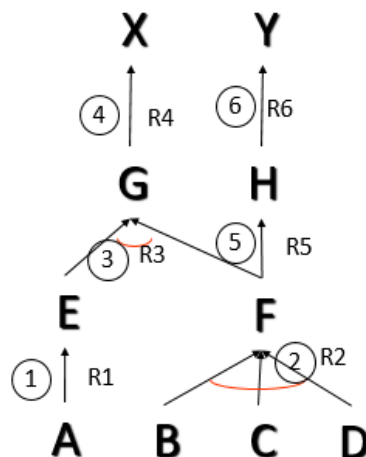
8. Priority / Saliency الأولوية أو الأهمية

- كل قاعدة قد تحمل مستوى أولوية معين (رقم أو تصنيف).
- القواعد ذات الأولوية الأعلى تُنفَّذ أولاً، بغض النظر عن موقعها النصي.

حينما يكون هناك أكثر من قاعدة قابلة للتنفيذ، يستخدم النظام معايير مثل: الترتيب النصي، الحداثة، البساطة، التخصص، وغيرها لاختيار القاعدة الأنسب للتفعيل. من المهم جداً فهم هذه المعايير عند تصميم أنظمة خبيرة فعالة وسريعة الاستجابة.

مثال 2:

Textual Order & Refractoriness Example



R1: IF A THEN E
 R2: IF B & C & D THEN F
 R3: IF E & F THEN G
 R4: IF G THEN X
 R5: IF F THEN H
 R6: IF H THEN Y

- assume **refractoriness** is used with **textual order**

القاعدة الأساسية: عندما نستخدم استراتيجية الترتيب النصي، فإن النظام يبحث عن القواعد بالترتيب كما هي مكتوبة (من الأعلى إلى الأسفل)، وينفذ أول قاعدة ينطبق شرطها.
 لكن عند استخدام الجمود (Refractoriness)، فإن القاعدة لا تنفذ مرة أخرى بعد تنفيذها لمرة واحدة (أو خلال دورة معينة)، حتى لو ظل شرطها متحققاً. الهدف هو منع تكرار غير مفيد أو حلقة لانهائية.

فترض أن الحقائق الأولية (Initial Facts):

A = true

B = true

C = true

D = true

التحليل خطوة بخطوة مع استخدام Textual Order + Refractoriness

1. الدورة الأولى (Cycle 1):



تحقق الشروط:

- R1: ✓ A موجود → تفعّل → نضيف E
- R2: ✓ B, C, D موجودة → تفعّل → نضيف F

قاعدة مفعلة:

- R2 و R1

قاعدة المعرفة الآن:

- A, B, C, D, E, F

ملاحظة R1 و R2 أصبحتا "خامدتين" (Refractory) ولن تُفعّل ثانية في الدورة القادمة.

2. الدورة الثانية (Cycle 2):

تحقق الشروط:

- R3: ✓ E, F موجودان → تفعّل → نضيف G
- R5: ✓ F موجود → تفعّل → نضيف H

قاعدة مفعلة:

- R5 و R3

قاعدة المعرفة الآن:

- A, B, C, D, E, F, G, H

3. الدورة الثالثة (Cycle 3):

تحقق الشروط:

- R4: ✓ G موجود → تفعّل → نضيف X



■ H : R6 موجود → تَفْعَل → نضيف Y

قاعدة مفعلة:

■ R6 و R4

قاعدة المعرفة الآن:

■ A, B, C, D, E, F, G, H, X, Y

4. الدورة الرابعة (Cycle 4) :

لا توجد قواعد قابلة للتفعيل:

■ جميع القواعد إما غير محققة شروطها، أو قد تم تفعيلها مسبقاً وأصبحت خامدة (Refractory) .

النظام يتوقف هنا، لأن جميع القواعد تم تفعيلها أو لا يمكن تفعيلها مجدداً.

ملخص التنفيذ حسب كل دورة:

القواعد المفعلة	الدورة	الحقائق المضافة
R1, R2	1	E, F
R3, R5	2	G, H
R4, R6	3	X, Y
—	4	(لا شيء - النظام يتوقف)

ما أهمية Refractoriness هنا؟

- بدون الجمود (Refractoriness) ، كانت R1 ستَفْعَل في كل دورة لأن A ما زال موجوداً.
- هذا كان سيؤدي إلى تكرار إضافة E ثم إعادة تنفيذ $R3 \rightarrow G \rightarrow X$ وهكذا في حلقة لا تنتهي.
- Refractoriness يمنع ذلك، ويجعل النظام يتقدم للأمام فقط.

Recency

الحادثة هي إحدى استراتيجيات حل التعارض في أنظمة القواعد، تستخدم عندما تكون هناك أكثر من قاعدة قابلة للتفعيل في نفس اللحظة (أي أن شرط "IF" في أكثر من قاعدة تحقق). تقوم هذه الاستراتيجية باختيار القاعدة التي تعتمد على البيانات الأحدث زمنياً في قاعدة البيانات.

مبدأ العمل:

1. يتم (Tagging) كل حقيقة (Fact) في قاعدة البيانات بختم زمني (Time Stamp) عند إضافتها.
2. عندما يتم تفعيل عدة قواعد (Conflict Set) ، ينظر النظام إلى تواريخ البيانات المستخدمة في شروط كل قاعدة.
3. تفضل القاعدة التي تعتمد على البيانات التي أضيفت مؤخراً.
4. إذا كانت قاعدتان تستخدمان نفس المعلومة الأحدث، يُقارن توقيت البيانات الأخرى المستخدمة في كل قاعدة لتحديد من الأجدر بالتنفيذ.

مثال توضيحي:

قاعدة البيانات تحوي الحقائق التالية:

الوقت المضاف	الحقيقة
08:16 مساءً – 1996/11/25	forecast is rain
10:18 صباحاً – 1996/11/26	weather is wet

ولدينا القاعدتان:

Rule1:

IF forecast is rain
THEN advice is "take an umbrella"

Rule 2:

IF weather is wet
THEN advice is "stay home"



ما الذي يحدث هنا؟

- كلا القاعدتين شروطهما متحقق.
- لكن:

- "forecast is rain" أضيفت ليلة البارحة.
- "weather is wet" أضيفت صباح اليوم.

إذاً، حسب استراتيجية Recency، يتم تفعيل القاعدة 2 لأنها تعتمد على معلومة أُضيفت في وقت أقرب (أحدث).

متى تكون هذه الاستراتيجية مفيدة؟

- في الأنظمة الخبيرة الزمنية أو التفاعلية، حيث يتم تحديث قاعدة البيانات باستمرار.
- مثال: نظم مساعدة الأطباء، نظم مراقبة الطقس، روبوتات الاستجابة السريعة.

Refractoriness (Breadth Strategy)

ماذا تعني Breadth ؟

- في حال وجود عدة قواعد قابلة للتنفيذ (Conflict Set)، تفضّل القواعد التي تطابق حقائق أقدم (أي تم تفعيلها منذ وقت طويل).
- بخلاف استراتيجية Recency التي تفضّل الأحدث، فإن Breadth تفضّل الأقدم.
- هذه الاستراتيجية تدفع النظام نحو استكشاف الفروع الأفقية أولاً (Breadth-First Search) بدلاً من الغوص مباشرة في عمق السلسلة.

المثال:

القواعد:



R1: IF A THEN E

R2: IF B & C & D THEN F

R3: IF E & F THEN G

R4: IF G THEN X

R5: IF F THEN H

R6: IF H THEN Y

الحقائق الأولية:

A = true

B = true

C = true

D = true

Refractoriness مفعّل: أي أن القاعدة لا تنفذ أكثر من مرة.

التحليل خطوة بخطوة مع Breadth + Refractoriness

1. الدورة الأولى (Cycle 1):

القواعد الممكنة:

■ R1: لأن A موجود → نضيف E

■ R2: لأن B و C و D موجودة → نضيف F

تُفضل جميع القواعد لأن ليس هناك تمييز زمني بعد (جميع الشروط قديمة بنفس الدرجة).

❖ النظام ينفذ R1 ثم R2 حسب ترتيبها النصي (لأنهما متساويتان زمنيًا).

قاعدة المعرفة الآن:

■ A, B, C, D, E, F

2. الدورة الثانية (Cycle 2):

القواعد الممكنة:

■ R3: IF E & F THEN G

■ R5: IF F THEN H

■ الآن لدينا تنافس بين R3 و R5 .

■ كلاهما يعتمد على F (جديد).

■ لكن:

■ R3 تعتمد أيضًا على E ، وهو أقدم من F .

■ R5 تعتمد فقط على F (الأحدث).

حسب استراتيجية Breadth نفضل القاعدة التي تعتمد على الأقدم.

✓ R3 تعتمد على E (أقدم من F) ، تنفذ أولاً ، ثم تنفذ R5 .



قاعدة المعرفة الآن:

A, B, C, D, E, F, G, H

3. الدورة الثالثة (Cycle 3) :

القواعد الممكنة:

R4: IF G THEN X

R6: IF H THEN Y

مقارنة:

G و H أضيفا في نفس الدورة.

R4 تعتمد على G

R6 تعتمد على H

لا تفضيل واضح زمنياً، نعود للترتيب النصي.

R4 ثم R6 ✓

قاعدة المعرفة الآن:

A, B, C, D, E, F, G, H, X, Y

4. الدورة الرابعة:

لا توجد قواعد قابلة للتفعيل. كل القواعد نفذت وتم كبحها (refractory) .

ملخص تنفيذ Breadth Strategy :

الدورة	القواعد المنفذة	سبب التفضيل
1	R1, R2	بداية، لا فرق زمني
2	R3, R5	R3 تعتمد على E الأقدم
3	R4, R6	متساويتان، نعود للترتيب النصي
4	—	جميع القواعد خامدة (refractory)

Specificity Strategy

وتعرف أيضاً بـ: Longest matching strategy / maximum specificity

هي إحدى استراتيجيات حل التعارض، وتعتمد على درجة خصوصية القاعدة، أي:

كلما كانت القاعدة أكثر تحديداً (أي تحتوي على عدد أكبر من الشروط)، كانت أكثر أولوية للتنفيذ.

المبدأ الأساسي:

إذا كانت الجهة اليسرى (LHS) لقاعدة ما عبارة عن مجموعة شاملة لجهة يسار قاعدة أخرى، فإنها تعتبر أكثر خصوصية.

القاعدة الأكثر خصوصية تستخدم معلومات أكثر → تفيد في الحالات الخاصة والاستثناءات.

لذلك، يُفَضَّل تنفيذ القواعد الأكثر خصوصية أولاً، لأن القواعد العامة قد تُطبَّق حتى في غير محلها إذا لم تُفعل الخصوصية.

مثال:

لدينا القاعدتان:



Rule 1:

IF the season is autumn
AND the sky is cloudy
AND the forecast is rain
THEN the advice is "stay home"

Rule 2:

IF the season is autumn
THEN the advice is "take an umbrella"

تحليل القواعد:

- القاعدة 1 تحتوي على 3 شروط $\Rightarrow$ أكثر تحديداً
- القاعدة 2 تحتوي على شرط واحد فقط $\Rightarrow$ أقل تحديداً

السيناريو الأول:

قاعدة المعرفة:

the season is autumn
the sky is cloudy
the forecast is rain

- كلا القاعدتين قابلة للتفعيل.
- لكن القاعدة 1 تحتوي على كل شروط القاعدة 2 + شروط إضافية.
- حسب استراتيجية Specificity نُنفذ القاعدة 1 لأنها أكثر تحديداً وتعالج الحالة بشكل أعمق.

السيناريو الثاني:

قاعدة المعرفة:

the season is autumn

- فقط القاعدة 2 قابلة للتفعيل (لأن باقي شروط القاعدة 1 غير موجودة).
- في هذه الحالة: القاعدة 2 تُنفذ.

الخاصية	القاعدة 1	القاعدة 2
عدد الشروط (LHS)	3 شروط	1 شرط
درجة الخصوصية	عالية (حالة خاصة)	منخفضة (حالة عامة)
التفعيل عند وجود كل المعطيات	تُنفذ أولاً	تُستبعد مؤقتاً
التفعيل عند نقص المعطيات	لا تُفعل لعدم توفر الشروط	تُنفذ

تساعد استراتيجية Specificity على إعطاء الأولوية للقاعدة الأكثر تفصيلاً، مما يجعل النظام الخبير أكثر دقة وواقعية، ويجنبه اتخاذ قرارات مبنية على قواعد عامة في حالات استثنائية.

استراتيجية البساطة (Simplicity Strategy)

أو: الأقل تعقيداً = الأكثر أولوية

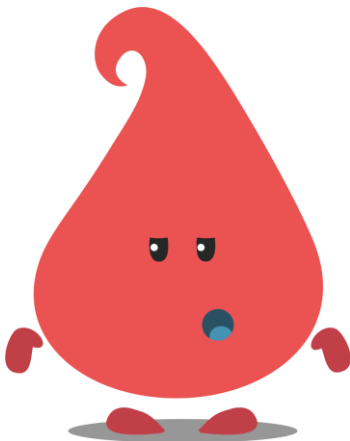
في بعض الأنظمة الخبيرة، عندما تتعدد القواعد الممكن تنفيذها (Conflict Set)، يتم تفضيل القواعد الأبسط، أي: القاعدة التي تحتوي على أقل عدد من الشروط (antecedents) في الجهة اليسرى (LHS) هي التي تُنفذ أولاً. خلاصة هذه الاستراتيجية:

- القاعدة الأبسط = أقل تكلفة حسابية + أكثر عمومية.
- غالباً ما تكون مناسبة كبدائية في اتخاذ القرار، ثم تتبع بقواعد أعقد إذا لزم الأمر.
- عكس استراتيجية Specificity التي تفضل القواعد الأكثر تعقيداً.

مثال:

القواعد:

- R1: IF A THEN E ← شرط واحد
 R2: IF B & C & D THEN F ← 3 شروط
 R3: IF E & F THEN G ← شرطين
 R4: IF G THEN X ← شرط واحد
 R5: IF F THEN H ← شرط واحد
 R6: IF H THEN Y ← شرط واحد



الحقائق الأولية:

A = true
 B = true
 C = true
 D = true

التحليل خطوة بخطوة باستخدام Simplicity + Refractoriness

نفترض أن Refractoriness مفعلة، أي لا يُعاد تنفيذ القاعدة نفسها مرتين.

1. الدورة الأولى:

القواعد القابلة للتنفيذ:

■ R1: A موجود → شرط واحد فقط

■ R2: B, C, D موجودة → 3 شروط

حسب Simplicity، نفضل القاعدة ذات الشروط الأقل. إذًا، R1 تنفذ أولاً.

❖ قاعدة المعرفة تصبح: A, B, C, D, E

الآن R2 ما زالت قابلة للتنفيذ، تنفذ R2 الوحيدة المتبقية.

❖ قاعدة المعرفة تصبح: A, B, C, D, E, F



2. الدورة الثانية:

القواعد القابلة للتنفيذ:

■ R3: E & F → 2 شروط

■ R5: F → شرط واحد

R5 أبسط → تنفذ أولاً.

قاعدة المعرفة تصبح: A, B, C, D, E, F, H

ثم تنفذ R3.

قاعدة المعرفة تصبح: A, B, C, D, E, F, H, G

3. الدورة الثالثة:

القواعد القابلة للتنفيذ:

■ R4: G → شرط واحد

■ R6: H → شرط واحد

كلاهما بسيطان (نفس عدد الشروط)

نلجأ إلى الترتيب النصي

R4 أولاً ثم R6

قاعدة المعرفة تصبح: A, B, C, D, E, F, H, G, X, Y

جدول التنفيذ:



الدورة	سبب التفضيل	القواعد المنفذة
1	R1 أبسط (شرط واحد مقابل ثلاثة)	R1 ثم R2
2	R5 أبسط (شرط واحد مقابل اثنين)	R3 ثم R5
3	متساويان، يُفصل حسب الترتيب	R4 ثم R6

تعتمد استراتيجية Simplicity على تفضيل القواعد ذات البنية الأبسط - أي التي تحتوي على عدد أقل من الشروط - وذلك لتقليل التعقيد وزيادة سرعة التنفيذ. تُستخدم هذه الاستراتيجية عادةً في الأنظمة التي تسعى لقرارات أولية سريعة، أو عندما تكون القواعد البسيطة كافية للتصرف في أغلب الحالات".

مقارنة Simplicity مع Specificity :

العنصر	Simplicity البساطة	Specificity التعقيد
التفضيل	الأقل شروطاً	الأكثر شروطاً
التركيز على	الحالات العامة	الحالات الخاصة والاستثناءات
نمط البحث	نحو السطح (Breadth/Simple)	نحو العمق (Depth-First)

استراتيجية LEX (Lexicographic Strategy)

الفكرة الأساسية:

LEX تستخدم عندما توجد أكثر من قاعدة قابلة للتنفيذ (Conflict Set). وهي تعتمد على 3 مراحل متسلسلة لحل التعارض:

1. Refraction التكرار ممنوع:
 - لا يُسمح للقاعدة نفسها أن تُنفذ مرتين.
 - هذا يمنع النظام من الدوران في حلقة مفرغة.
2. Recency الأحدث أولاً:
 - تُفضّل القاعدة التي تعتمد على المعطيات التي أُضيفت مؤخراً (وفق طوابع زمنية).
3. Specificity الأكثر تعقيداً:
 - إذا تساوت القواعد في الزمن، نختار القاعدة التي تحتوي على شروط أكثر (أي أكثر تخصيصاً).

مثال:

القاعدتان:

R1: IF the forecast is rain AND the sky is cloudy THEN advice is 'take umbrella'

R2: IF the sky is cloudy AND the forecast is rain THEN advice is 'take umbrella'

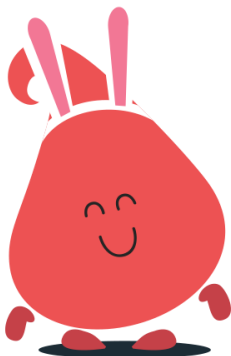
في LEX ، ترتيب الشروط لا يؤثر، أي:

$$E1 \text{ AND } E2 = E2 \text{ AND } E1$$

قواعد متطابقة:

- إن كانت R1 نفذت سابقاً، لن تُنفذ مرة أخرى بسبب Refraction.
- لو لم تنفذ، فيتم النظر في تاريخ إدخال المعطيات:
 - ❖ إذا forecast أُضيف بعد R1 → sky تُنفذ.
 - ❖ إذا تساوى → ننتقل إلى Specificity ، ثم إلى الترتيب في قاعدة المعرفة.

خطوات تطبيق LEX :



1. تطبيق Refraction : حذف القواعد المنفذة مسبقاً.
2. تطبيق Recency : اختيار القاعدة التي تعتمد على أحدث البيانات.
3. تطبيق Specificity : اختيار القاعدة ذات الشروط الأكثر.
4. إن استمر التعادل : نعود إلى الترتيب النصي أو الأولوية (Salience) .

استراتيجية الأولوية (Priority / Salience)

التعريف:

كل قاعدة تُمنح رقماً يُمثل أولويتها (Salience) ، ويُعبّر هذا الرقم عن:

- ❖ مدى أهمية القاعدة في اتخاذ القرار.
- ❖ القاعدة ذات الرقم الأعلى تُنفذ أولاً.

كيف نستخدم الأولوية؟

- في الأنظمة البسيطة ($100 \geq$ قاعدة): يمكن ترتيب القواعد يدوياً في قاعدة المعرفة.
- في الأنظمة المعقدة: يجب وضع أرقام Salience لكل قاعدة لتوضيح أولويتها.

متى نستخدم الأولوية؟

- عندما نريد تنفيذ قواعد "مهمة" أولاً (مثل إنقاذ الأرواح).
- أو تأجيل قواعد غير حرجية حتى يتم تنفيذ ما هو أهم.

مثال:

Rule 1:

IF patient has pain , THEN prescribe painkillers priority 10.

Rule 2:

IF patient has chest pain, THEN treat for heart disease priority 100.

- ومع ذلك طرق حل التضارب المذكورة أعلاه بسيطة وسهلة التنفيذ. في معظم الحالات تنفر هذه الطرق حلولاً مرضية.
- البرنامج وتعقيده، يصبح من الصعب بشكل متزايد على مهندس المعرفة إدارة القواعد والإشراف عليها في قاعدة المعرفة.
- يجب أن يتحمل نظام الخبراء نفسه بعض العبء على الأقل أو أن يفهم سلوكه الخاص.

المعرفة الفوقية (Meta-Knowledge)

وهي المعرفة حول المعرفة، أي معرفة كيف يجب استخدام القواعد، ومتى، ولماذا تفضل قاعدة على أخرى. في الأنظمة الخبيرة، يتم تمثيل المعرفة الفوقية عبر Meta-Rules، وهي قواعد تتحكم في استخدام القواعد الأخرى.

لماذا نحتاجها؟

كلما كبرت قاعدة المعرفة، أصبح من الصعب:

- التحكم في ترتيب القواعد يدوياً.



تحديد أفضل قاعدة لكل موقف.

لذا، نُعلِّم النظام أن يتحكم في نفسه عبر Meta-Rules.

مثال:

Metarule 1:

القواعد القادمة من خبراء ← لها أولوية أعلى من القواعد التي وضعها مبتدئون.

Metarule 2:

القواعد الخاصة بإنقاذ الأرواح ← تُنفَّذ قبل القواعد الخاصة بصيانة المعدات.

هل يمكن لنظام خبير فهم واستخدام قواعد التعريف (meta rules)؟ توفر بعض أنظمة الخبراء محرك استدلال منفصل كـ meta rules.

لا تستطيع معظم أنظمة الخبراء التمييز بين القواعد (rules) وقواعد التعريف (meta rules)

يجب إعطاء قواعد التعريف الأولوية القصوى في قاعدة المعرفة الحالية.

عند تشغيلها، تقوم قاعدة التعريف "بحقن" بعض المعلومات المهمة في قاعدة البيانات والتي يمكن أن تغير أولويات بعض القواعد الأخرى.

الخلاصة:

المفهوم	التعريف
LEX	استراتيجية حل تعارض بثلاث مراحل Refraction → Recency → Specificity
Priority (Saliency)	كل قاعدة تُعطى رقماً للأهمية، تُنفذ الأعلى أولاً
Meta-Knowledge	قواعد تتحكم في القواعد الأخرى لتوجيه النظام حسب منطق أعلى وأذكى

انتهت المحاضرة

